Informe de la primera entrega: Sistema distribuido basado en servicios para la reserva de boletos

Alejandro Martínez Mesa, Julian Rojas Almanza y Samuel Freile Avendaño

Pontificia Universidad Javeriana, Facultad de Ingeniería, Departamento de Ingeniería de Sistemas

Introducción a Sistemas Distribuidos

John Corredor Franco

11 de octubre de 2026

Tabla de contenido

1. Introducción 3
2. Modelos del sistema 3
2.1 Modelo arquitectónico 3
2.2 Modelo de interacción 3
2.3 Modelo de fallos 3
2.4 Modelo de seguridad 3
3. Diseño de la arquitectura base 4
3.1 Despliegue 4
3.2 Componentes 4
3.3 Clases 4
3.4 Secuencia 5
4. Componentes para la escalabilidad y la tolerancia a fallas 6
5. Protocolo de pruebas 6
6. Herramientas y estrategias para las métricas de desempeño 7
7. Generación de carga 7
Referencias 8
Anexo. Repositorio del proyecto 8

Tabla de figuras

Figura 1. Diagrama de despliegue de la arquitectura base 4
Figura 2. Diagrama de componentes de la arquitectura base 4
Figura 3. Diagrama de clases del sistema 5
Figura 4. Diagrama de secuencia de la reserva y la cancelación 6

Lista de tablas

Tabla 1. Componentes previstos y su justificación 6
Tabla 2. Protocolo de pruebas para la entrega final 6

Informe de la primera entrega: Sistema distribuido basado en servicios para la reserva de boletos

1. Introducción

En este informe se describe el diseño del sistema de reserva de boletos (SRB) de la Universidad de las Américas, que permite a más de 10 000 estudiantes consultar eventos, reservar, modificar y cancelar. En esta entrega se construyó la arquitectura base de tres capas con la consulta y la reserva, y se dejó diseñado todo el sistema de la Figura 1 del enunciado. Se ha programado en Java 21 con JeroMQ, la versión de ZeroMQ para Java que permite a los procesos enviarse mensajes por la red (Hintjens, 2013); los datos se han guardado en PostgreSQL y todo se ha desplegado con Docker Compose.

2. Modelos del sistema

2.1 Modelo arquitectónico

Es un sistema cliente-servidor en tres capas (Coulouris et al., 2012), con un proceso de Java por servicio: el Gestor de Transacciones, el Servicio de Procesamiento de Reservas, Modificaciones y Consultas (en adelante, servicio de procesamiento), el de cancelaciones y el de persistencia. Los servicios de negocio no guardan nada: todo queda en PostgreSQL, y solo la persistencia habla con la base de datos. Los procesos se mandan mensajes JSON por ZeroMQ: petición y respuesta (Request/Reply, quien pregunta espera la respuesta) para consulta, reserva y modificación, y envío y recepción (Push/Pull, quien envía no espera) para la cancelación. El Gestor de Transacciones es la puerta de entrada, así que los clientes no conocen el resto del sistema.

2.2 Modelo de interacción

El sistema es asíncrono, como plantea el enunciado: no se conoce cuánto tarda un mensaje ni qué tan rápido va cada proceso. Por eso cada salto tiene un tiempo máximo de espera, más corto cuanto más adentro: 5 segundos en el cliente, 4 del gestor al servicio de procesamiento y 3 de este a la persistencia. Consulta, reserva y modificación son síncronas; la cancelación es asíncrona: el gestor la acepta enseguida y el servicio de cancelaciones la aplica después y avisa al cliente.

2.3 Modelo de fallos

Se consideran dos tipos de fallas: que un proceso se caiga y que un mensaje se pierda o llegue tarde. En esta entrega no se ha implementado tolerancia a fallas, como dice el enunciado: si un servicio se cae, nada lo reemplaza hasta que se vuelva a arrancar. Solo se ha evitado que la falla deje el sistema en mal estado: con el tiempo máximo de espera el cliente recibe un error en vez de quedarse esperando, y cada reserva es una transacción que se guarda completa o no deja nada. El gestor, el servicio de procesamiento y la base de datos son puntos únicos de falla, que se resolverán en la segunda entrega.

2.4 Modelo de seguridad

Se asume una red interna de confianza y se protegen las reservas de solicitudes mal armadas. El gestor revisa el formato y el tipo de cada campo; la persistencia escribe las consultas con parámetros (`PreparedStatement`), lo que evita que alguien meta código SQL en un texto; y la base de datos tiene restricciones (`CHECK`) para que los ocupados nunca superen los 100. Cada capa tiene su red y solo se abren los puertos necesarios. Aún no hay contraseñas ni cifrado; ZeroMQ trae CURVE para ambos.

3. Diseño de la arquitectura base

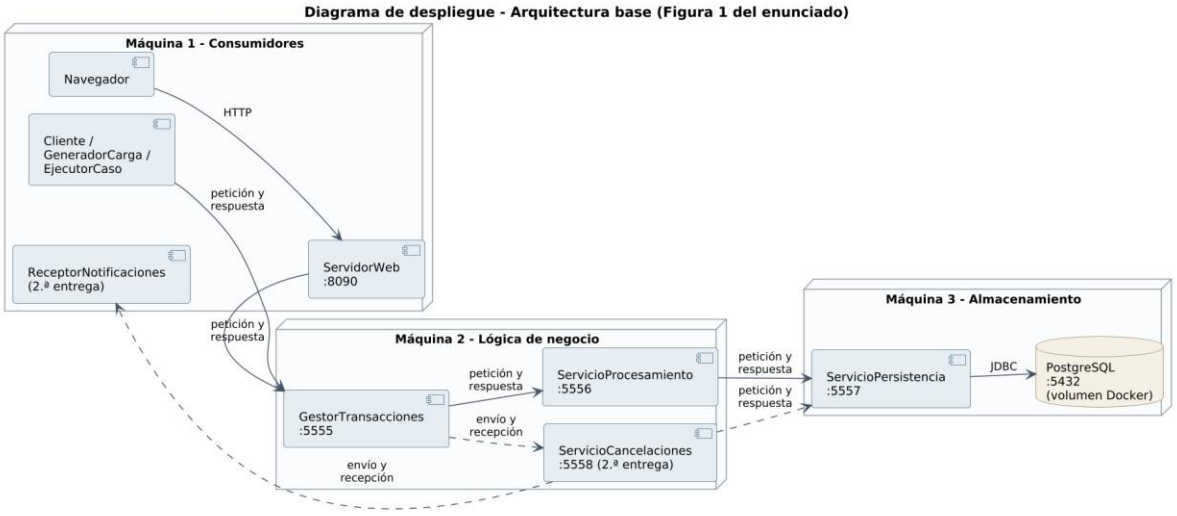
Se ha diseñado todo el sistema de la Figura 1 del enunciado, incluido el servicio de cancelaciones, aunque solo se han programado la consulta y la reserva.

3.1 Despliegue

Cada capa va en una máquina (Figura 1): la página web y los clientes en la máquina 1; el Gestor de Transacciones (puerto 5555), el servicio de procesamiento (5556) y el de cancelaciones (5558) en la 2; y el servicio de persistencia (5557) con PostgreSQL en la 3. Cada máquina tiene su archivo de Docker Compose y las tres comparten la carpeta del proyecto por NFS.

Figura 1

Diagrama de despliegue de la arquitectura base

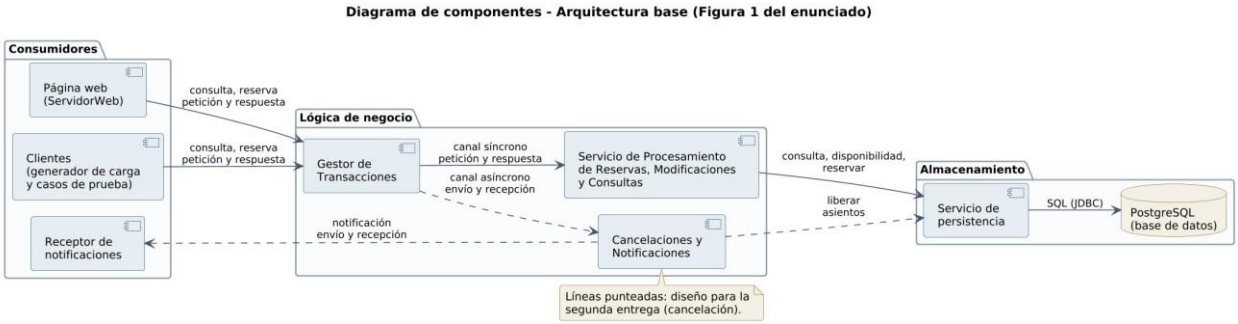


3.2 Componentes

El Gestor de Transacciones es la única entrada: manda las consultas, reservas y modificaciones al servicio de procesamiento (canal síncrono) y las cancelaciones al servicio de cancelaciones (canal asíncrono). Estos dos solo tocan los datos a través de la persistencia (Figura 2).

Figura 2

Diagrama de componentes de la arquitectura base



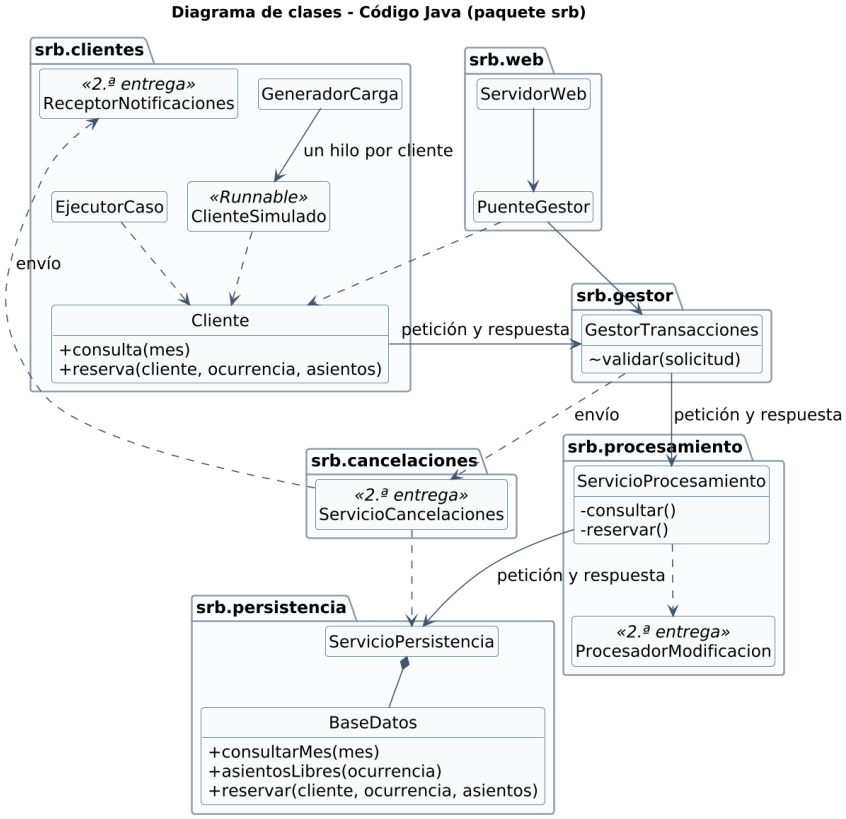
3.3 Clases

Todo el código está en el paquete `srb`, con un subpaquete por servicio (Figura 3). Cada servicio es una clase con su método `main`. `Util` agrupa lo que todos usan (configuración, bitácora

o registro de lo que pasa, y envío de mensajes con tiempo máximo) y `BaseDatos` guarda todo el SQL en un solo lugar. Las clases de la segunda entrega ya existen vacías, con un comentario que explica qué harán.

Figura 3

Diagrama de clases del sistema

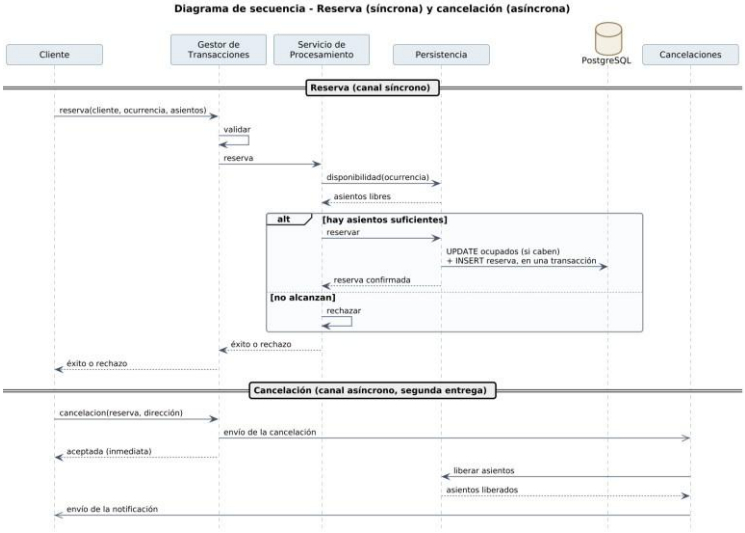


3.4 Secuencia

En una reserva (Figura 4) el servicio de procesamiento primero pregunta cuántos asientos libres hay y, si alcanzan, pide confirmar. El servicio de persistencia vuelve a revisar el cupo dentro de la misma transacción; así, aunque dos personas reserven al mismo tiempo, nunca se pasan de los 100 asientos. La consulta hace el mismo recorrido sin escribir.

Figura 4

Diagrama de secuencia de la reserva y la cancelación



4. Componentes para la escalabilidad y la tolerancia a fallas

Para la segunda entrega se agregarán seis componentes que atacarán los cuellos de botella (el servicio de procesamiento y la base de datos) y los puntos únicos de falla (el gestor y el servicio de procesamiento). Aquí solo se mencionan y justifican (Tabla 1).

Tabla 1

Componentes previstos y su justificación

Componente	Atributo	Justificación
Copias del procesamiento (`BalanceadorProcesamiento`)	Escalabilidad	Varias copias se reparten las solicitudes.
Réplica de PostgreSQL (`ReplicaBaseDatos`)	Escalabilidad	Atiende consultas y libera a la base principal.
Gestor de respaldo (`GestorRespaldo`)	Tolerancia a fallas	Toma el lugar del gestor si se cae.
Detección de caídas (`MonitorFallas`)	Tolerancia a fallas	Si un servicio no contesta, arranca otro.
Identificador único por solicitud	Tolerancia a fallas	Reintentar sin duplicar la operación.
Cola durable de cancelaciones	Tolerancia a fallas	Ninguna cancelación aceptada se pierde.

5. Protocolo de pruebas

Para la entrega final se probará el sistema desde cada función por separado hasta el sistema completo en tres máquinas (Tabla 2). Las pruebas automáticas usan JUnit 5, la herramienta más usada para probar programas en Java; ya se han escrito 59: 46 unitarias y 13 de integración, que usan la base de datos real y la dejan como estaba.

Tabla 2

Protocolo de pruebas para la entrega final

Tipo de prueba	Qué se comprueba	Herramienta	Estado
Unitarias	Reglas del gestor, mensajes y métricas	JUnit 5	Hecha

Tipo de prueba	Qué se comprueba	Herramienta	Estado
Integración	Base de datos y sistema completo	JUnit 5	Hecha
Funcionales y aceptación	La base final coincide con el caso de prueba	`EjecutorCaso`	Hecha
Concurrencia	Reservas simultáneas no pasan de 100 asientos	JUnit 5	Hecha
Validación	Datos mal escritos se rechazan sin cambios	JUnit 5	Hecha
Seguridad	Textos con código SQL no alteran la base	Manual	Final
Rendimiento	Tiempos y transacciones con 3, 5 y 8 clientes	Generador de carga	Final
Estrés	Hasta dónde aguanta con más clientes	Generador de carga	Final
Escalabilidad	Cuánto mejora la arquitectura escalable	Generador de carga	Final
Tolerancia a fallas	Apagar el gestor en plena carga sin perder nada	`docker stop`	Final
Despliegue	Que funcione con una capa por máquina	`deploy/`	Final
Usabilidad	Que la página se entienda sin explicación	Compañeros	Final
Regresión	Que lo que servía siga sirviendo	JUnit 5	Siempre

6. Herramientas y estrategias para las métricas de desempeño

Para cada escenario de la Tabla 1 del enunciado (3, 5 y 8 clientes) y cada operación (consulta y reserva) se medirán el tiempo promedio de respuesta, su desviación estándar (qué tanto se alejan los tiempos del promedio) y las transacciones por minuto. El tiempo de cada solicitud es la hora de recepción menos la de envío, tomada con `System.nanoTime()` de Java dentro de `GeneradorCarga`, que también calcula el promedio, la desviación y las transacciones por minuto (respuestas recibidas × 60 / duración en segundos) y agrega una fila por operación a `resultados/metricas.csv`. Además, el Gestor de Transacciones escribe en su bitácora, cada minuto, cuántas consultas y reservas atendió, que es la medida en el gestor que pide el enunciado (se lee con `docker compose logs`), y `docker stats` muestra la CPU y la memoria de cada contenedor. Antes de cada corrida se reiniciará la base de datos; cada escenario durará 2 a 3 minutos, se repetirá tres veces y se correrá en las mismas máquinas. Las tablas y los gráficos se harán con LibreOffice Calc.

7. Generación de carga

La carga la genera un programa propio en Java, `GeneradorCarga`. Cada cliente simulado es un hilo (una tarea que corre al mismo tiempo que las demás) que envía una solicitud al Gestor de Transacciones cada 60 / R segundos, donde R son las solicitudes por minuto. Los clientes «solo consulta» envían consultas; los demás mezclan consultas y reservas de 1 a 3 asientos. No se han usado Locust ni JMeter porque están hechos para páginas web (HTTP) y el sistema usa ZeroMQ. Con variables de entorno se cambian, sin tocar el código, el número de clientes y cuántos solo consultan, las solicitudes por minuto (mínimo 20), el porcentaje de consultas, la duración (2 a 3 minutos) y el mes. Así se reproducen los escenarios del enunciado: 3, 5 y 8 clientes, de los que 2, 3 y 4 solo consultan. Para un caso de prueba fijo, `EjecutorCaso` lee las operaciones de un CSV y las envía en orden, una por una.

Referencias

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). *Distributed systems: Concepts and design* (5th ed.). Addison-Wesley.

Hintjens, P. (2013). *ZeroMQ: Messaging for many applications*. O'Reilly Media.

Anexo. Repositorio del proyecto

El código fuente del sistema y el README con las instrucciones para desplegarlo en las tres máquinas se encuentran en el repositorio: https://github.com/Silverweta20/Proyecto-Boletas-Sistemas-Distribuidos